Version Control, Git, and GitHub

Ryan M. Richard

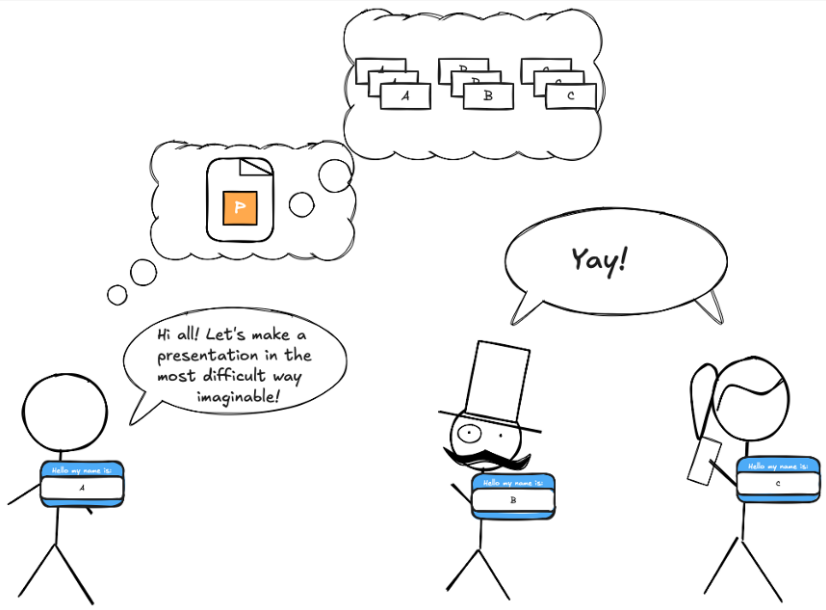
Scientist and Adjunct Assistant Professor of Chemistry
Ames National Laboratory and Iowa State University
Ames, IA, USA

2026 SIMCODES Bootcamp

Why Do We Need Version Control?

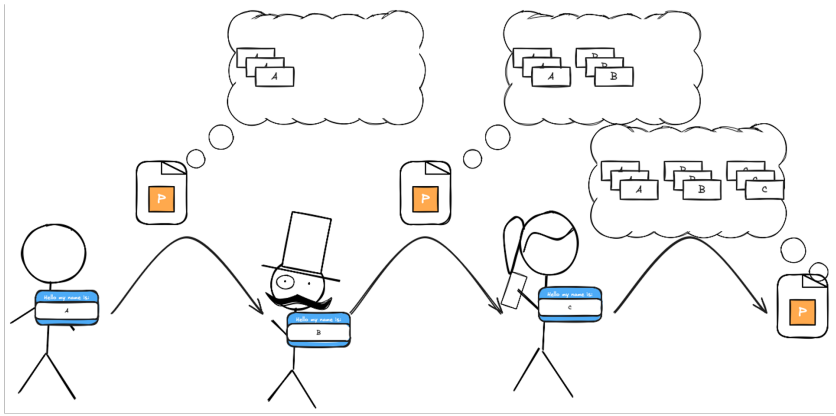
Motivation

- Imagine three people are trying to write this presentation.
- Assume their parents named them A, B, and C.
- Also assume these people haven't heard of Google Slides or Office 365 and are trying to do this via email.
- Assume the slides are supposed to be ordered so that A's slides come first, B's come second, and C's third.
- Let's work through some scenarios in gether.



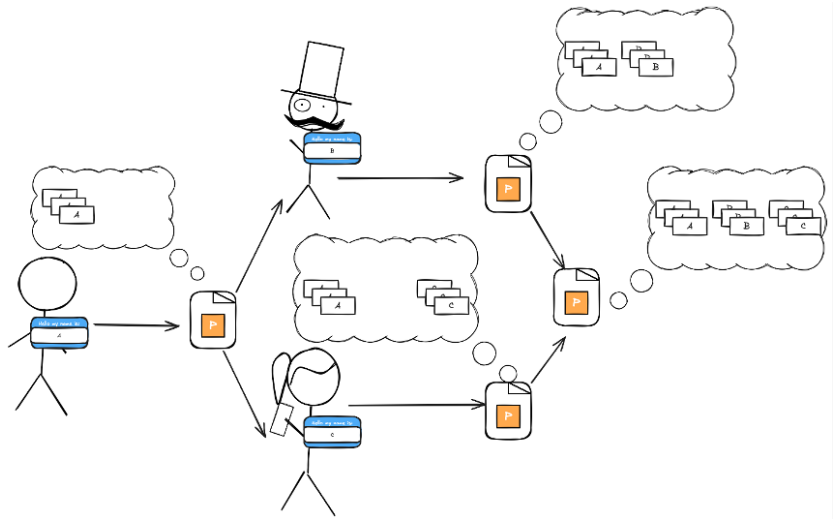
Scenario 1: Linear History (i.e., the history no time travel movie has)

- A makes their slides and emails the presentation to B.
- B adds their slides to the presentation A sends.
- B emails the presentation to C.
- C adds their slides.
- Presentation is done.
- Called a "linear history" because the timeline has no "branches."
  - "Branches" are easier to define when we have one.



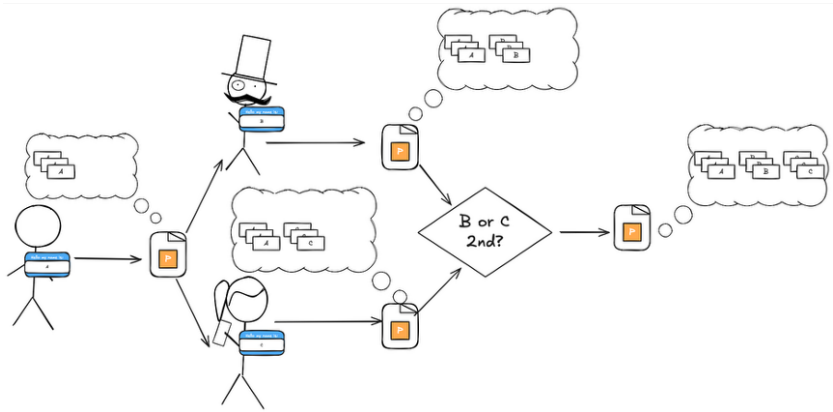
Scenario 2: Non-conflicting branched history (i.e., the timeline every time travel movie thinks it has)

- A makes their slides.
- A sends the slides to both B and C.
- Simultaneously:
  - B adds slides in the correct place.
  - C adds slides in the correct place.
- The two presentations are "magically" merged.
- "Branched history" because different changes happen simultaneously.



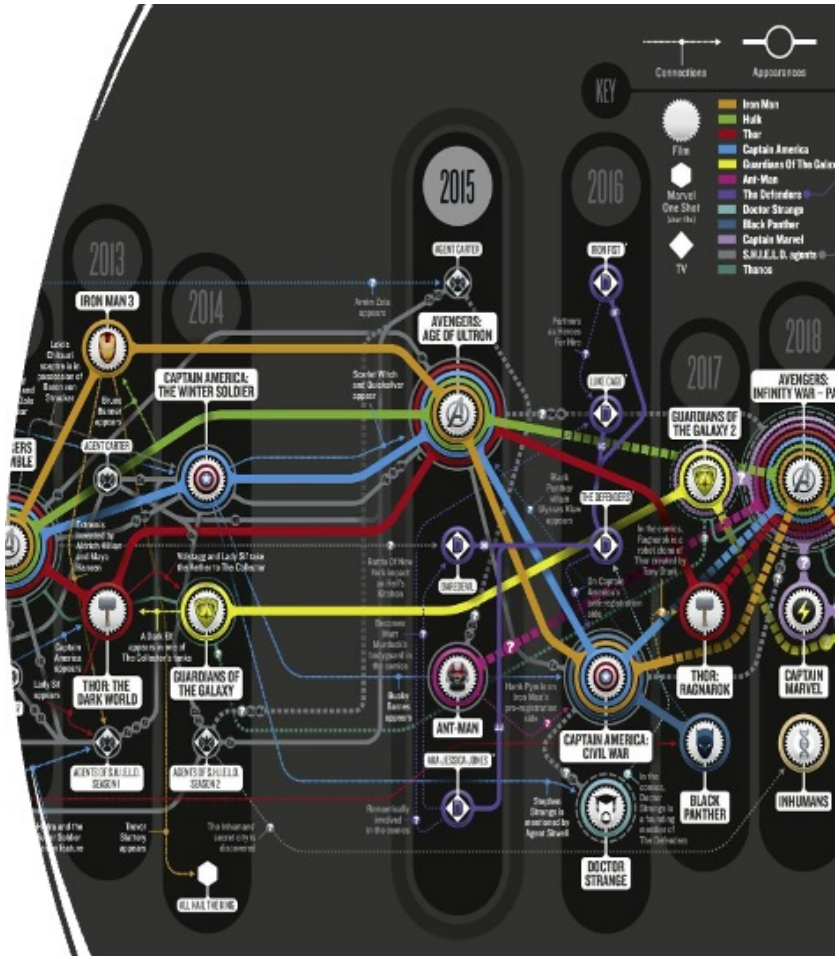
Scenario 3: The conflicting branched timeline (i.e., the timeline every time travel movie actually has)

- A makes their slides.
- Slides sent to both B and C.
- Simultaneously:
  - B appends their slides.
  - C appends their slides.
- Conflict: both B and C think their slides come directly after A.
- "Magic" merging moves C's slides to the end.
- "Branched history" because different changes happen simultaneously.
- "Conflicting" because the histories are not compatible.



Motivation Summary

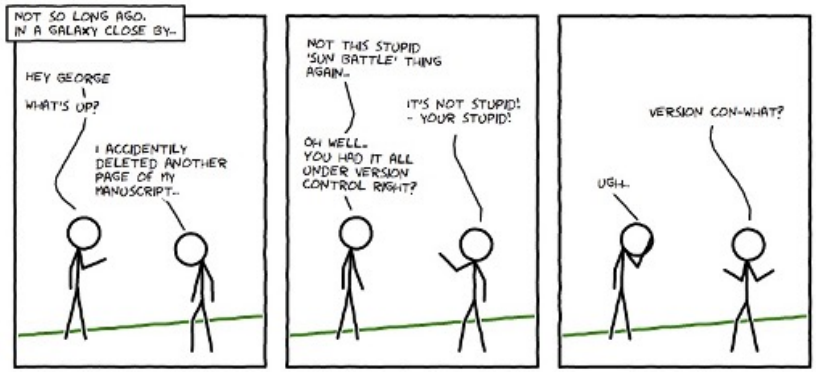
- Having multiple people work on the same file can be difficult without coordination.
- Guess what happens when we have multiple people work on multiple files without coordination?
- Modern software development almost always involves multiple people working on multiple files in an uncoordinated (or loosely coordinated) manner.



What is the Difference Among Version Control, Git, and GitHub?

What is Version Control (VC)?

- VC software is used to manage the history (i.e., versions) of a software project.
- Superficially, VC gives you save/load, undo, redo for a set of files.
- Automates (to the extent possible) the steps we termed "magic."
- Historically, there have been many different VC software packages.
- Now, most developers (95% according to Wikipedia) have coalesced around Git.



What is Git?

- Developed by Linus Torvalds for version controlling the Linux kernel.
  - Written because a proprietary VC software company revoked Linux community's free license.
  - Amusingly outperformed all other existing VC software after three weeks of development.
- According to Linus, git's meaning depends on your mood:
  - "Global information tracker" if you're in a good mood.
  - "Goddamn idiotic truckload of sh*t" when it breaks.

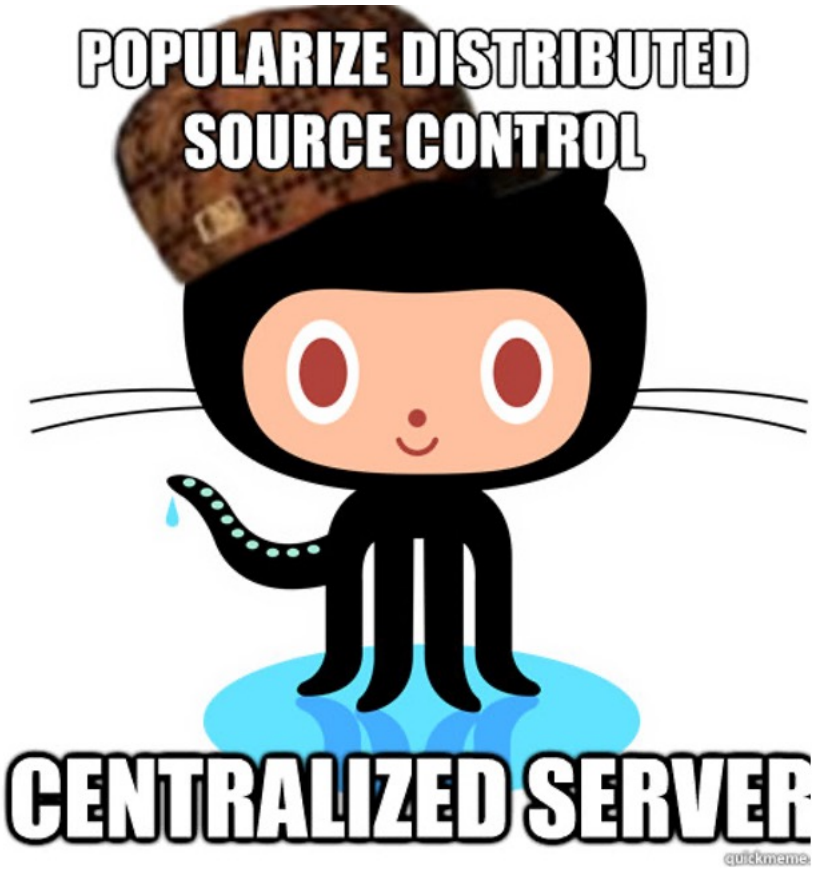
What is GitHub?

- For SIMCODES we are using Git in the context of GitHub.
- Git is the software responsible for VC-ing our software project.
- GitHub is "just" a storage solution (e.g., like Google Drive or DropBox).
- Use Git to interact with GitHub.

Git and GitHub Terminology

Disclaimer: Simplifications Ahead

- Git is a powerful VC solution.
- Power comes from not assuming any "sacred timeline."
- Leads to additional levels of abstraction that complicate things.
- Simplified by noting that the copy on GitHub is treated specially.
- The git Jedis in the room know that there's exceptions to pretty much everything I'm about to tell you.

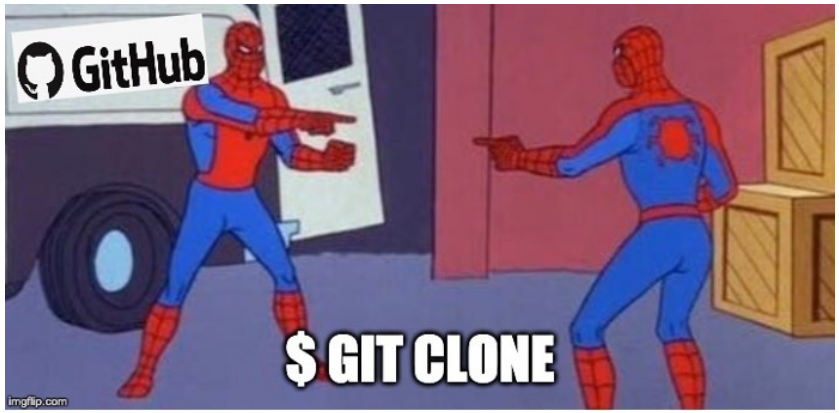


Term: Repository

- A **repository** contains the source files and the history of changes to those files.
- A **local** repository is a repository that lives on the computer you are currently using.
  - You always develop software in a local repository.
- A **remote** repository is a repository that lives on a different computer.
  - Assume "remote" is synonymous with "the repository that lives on GitHub".
- Repository is usually shortened to just "repo."

Term: Clone

- The first step in developing software is to get a copy of the existing source code.
- Usually, the repo you want to develop for already lives on GitHub.
- To get a local copy of a remote repo you **clone** it.
- Unless you delete your local repo, you typically only clone a repo once.



Term: Branch

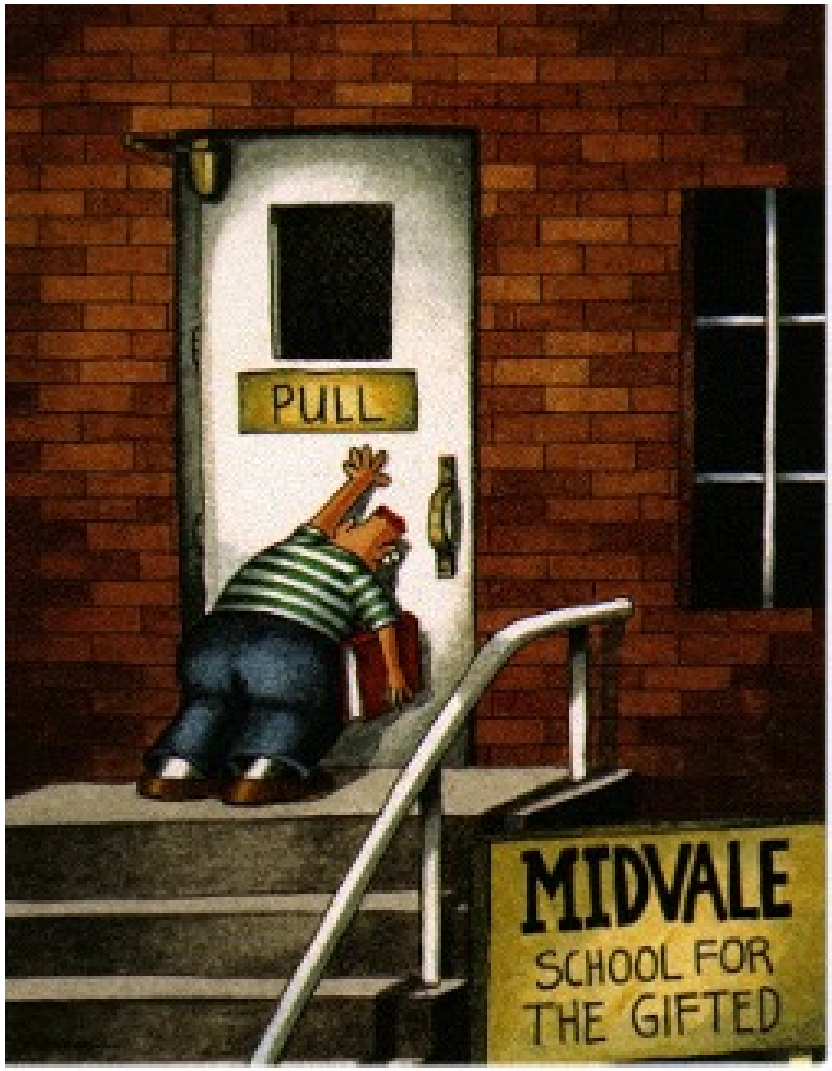
- A **branch** is a (linear) history of the repo.
  - It's one of the many timelines.
- Usually, one branch is special.
  - Often called the **main** or **master** branch.
- The special branch is the "final" version of the project.
- Other branches are sometimes called development branches.
- You should ALWAYS be working on a development branch.

Term: Commit

- Git doesn't come with "auto-save".
- A **commit** is the set of changes you made to the branch since the last commit.
  - Conceptually, a commit is Git's equivalent of "saving".
  - In the timeline analogy, commits are events.
- Commits can add content and/or remove content.
- Commits save your work on the current branch. Other branches remain unchanged.

Terms: Push, Pull, Merge

- When you are done developing a feature locally you need to add the feature to the remote repo.
- You **push** your changes from your local branch to the remote branch.
- You **pull** changes from a remote branch to your local branch.
- If you are moving changes from one local branch to another local branch you **merge** the branches.
- As a note, pushing and pulling are implemented in terms of merge, so "merge" is usually used as a catchall.



Term: Conflict

- Git is pretty good about merging changes automatically.
- However, when the same lines of code have been modified, Git usually needs help merging.
- When Git cannot automatically merge two branches, the branches are said to be in conflict.
- If a conflict occurs Git will show you the two changes and you will have to decide how to reconcile them.

Term: Organization

- Developer teams usually work on a series of related projects.
  - E.g., the SIMCODES team has one project for the website, one project for training materials, and several projects for research.
- Git provides no mechanism for grouping repositories (we don't speak of git submodule...).
- A GitHub **organization** allows a developer team to group their repos together.
- SIMCODES has the GitHub organization SIMCODES-ISU (SIMCODES was taken...).

Term: Pull Request

- A **pull request** (or PR for short) tells the maintainers of a GitHub repo that you want them to pull changes into their repo.
  - The repo owners can then decide if they want those changes or not.
- PRs are needed because we don'tt want just anyone making changes to our code.
- Most repos now require PRs, regardless of who is making changes.
  - PRs are used to automatically test changes before merging them.

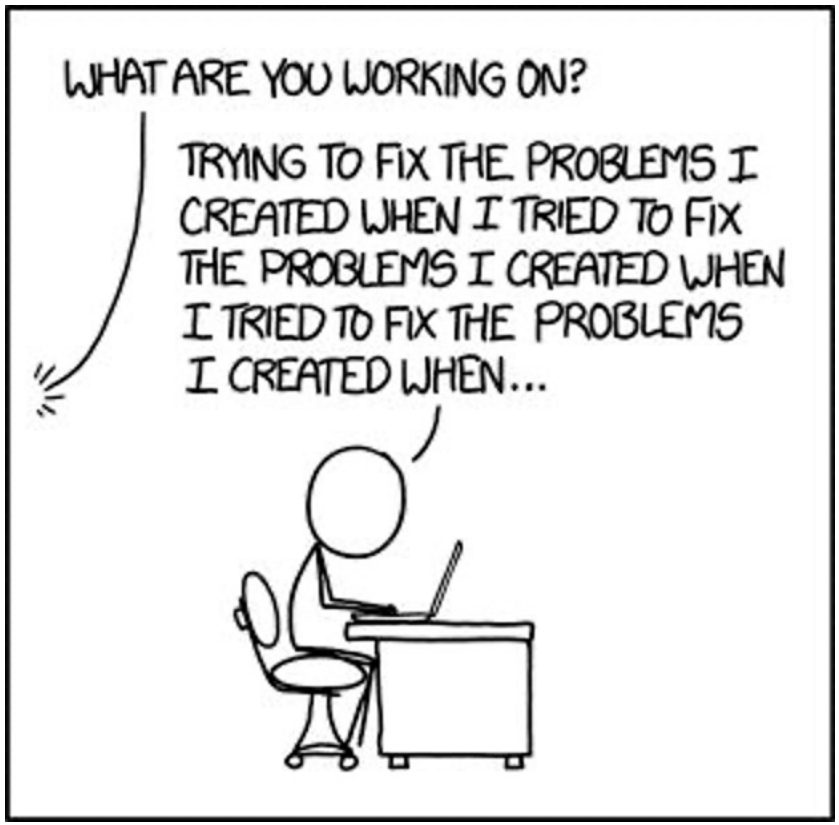
Term: Fork

- PRs must be between branches that live on GitHub.
- If you have sufficient privileges, then create a branch and make a PR.
  - But what if you don't have sufficient privileges?
- A GitHub **fork** is a clone of a GitHub repository that you own.
  - Can make the PR from a branch of your fork.
- Even if you have permission to create a branch in a repo, you can always fork the repo and make a PR from the fork.

Git/GitHub Workflow

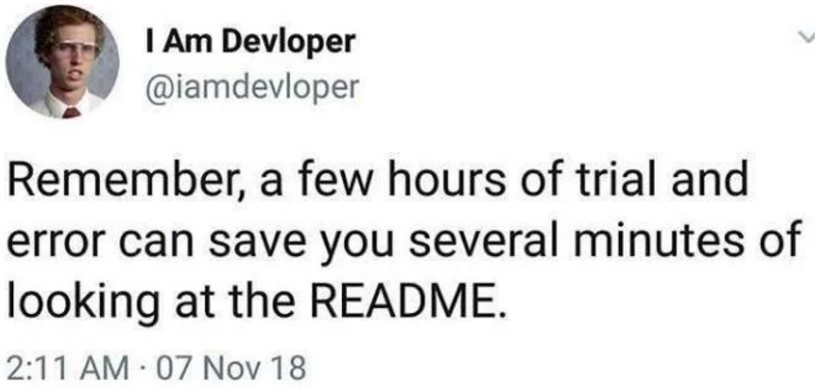
Public Service Announcement

- If you follow these steps, every single time you develop software you will rarely (if ever) encounter a Git/GitHub problem.
  - Disclaimer: problem is not the same as "conflict", though this workflow will greatly reduce conflicts too!
- If you don't follow these steps, you can still accomplish your goals, but you will make life 10x harder for yourself.
- In my experience, 99% of Git problems come from skipping one of theses steps.



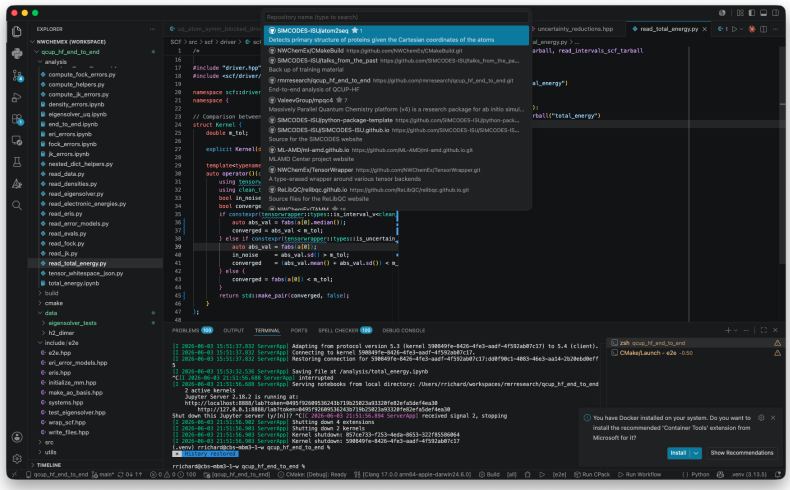
Hands-on Example

- We will make a small change to the README.md file.
- Look at the README.md file and identify a way to improve it.
  - Add the NSF acknowledgement.
  - Add a link to the documentation.
  - Add installation instructions.
  - Write "This is a change so I can learn GitHub."



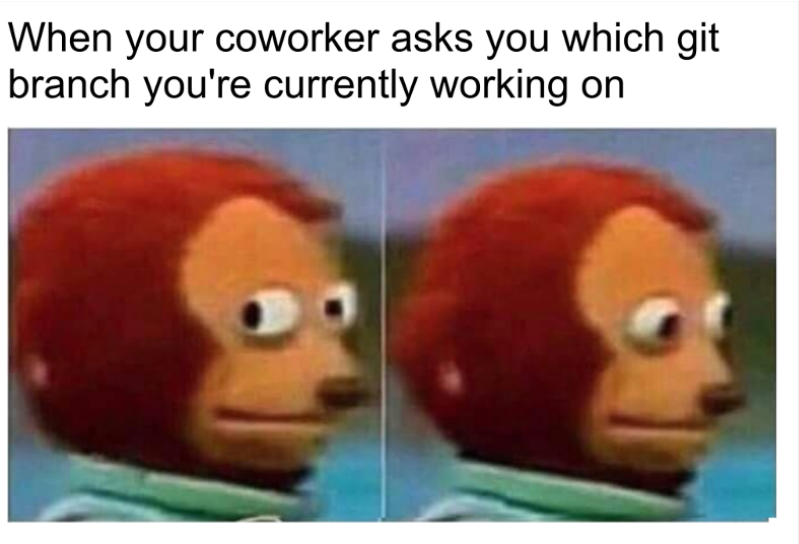
Step 0: Get a Repo

- Open command palette (ctrl+shift+P or cmd+shift+P).
- Type “git:clone”
- Select “Clone from github.”
- Pick your project or “SIMCODES-ISU/atom2seq”



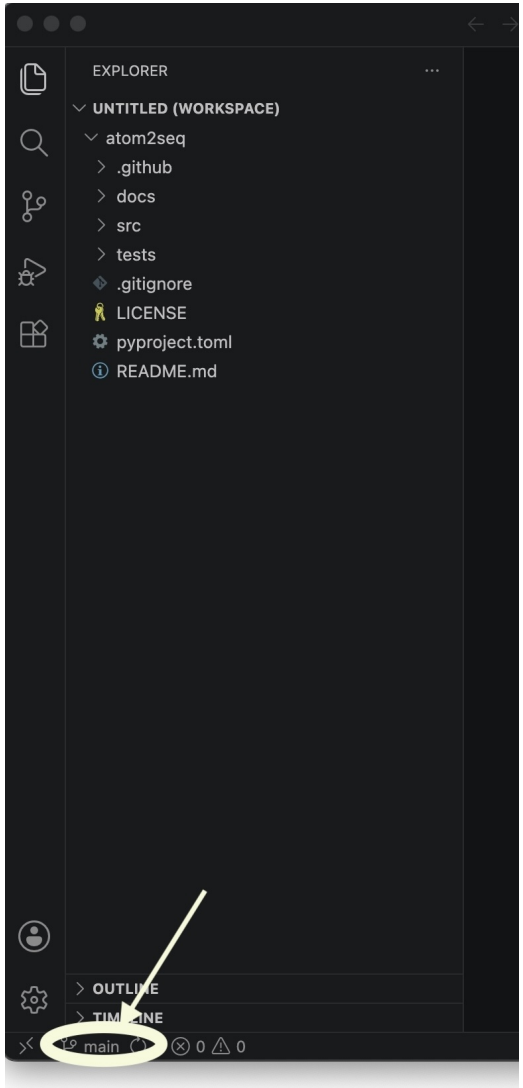
Step 1: Make a Feature Branch

- You should NEVER make changes to the main/master branch!
  - Exception: None. There is no exception!
  - But what about if... No! Don't do it!
- In my experience, making changes directly to the main/master branch is the primary way Git newbies get in a pickle.



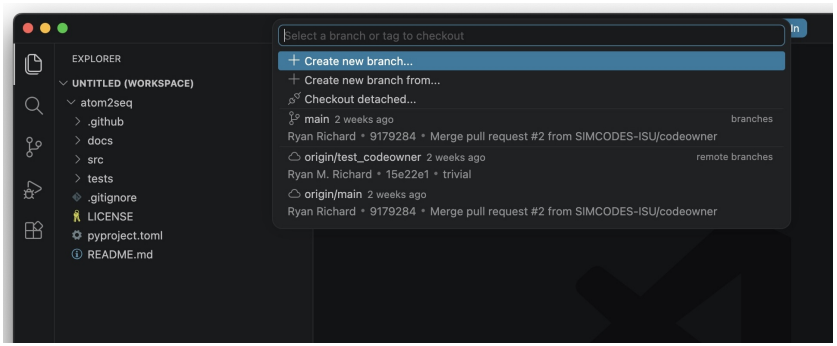
Step 1: Make a Feature Branch

- You should NEVER make changes to the main/master branch!
  - Exception: None. There is no exception!
  - But what about if... No! Don't do it!
- In my experience, making changes directly to the main/master branch is the primary way Git newbies get in a pickle.
- You start on the "main" branch.



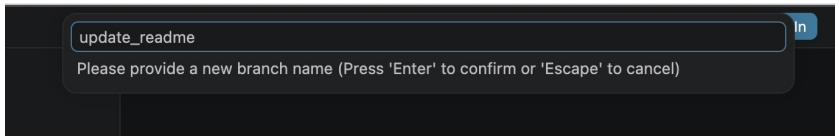
Step 1: Make a Feature Branch

- You should NEVER make changes to the main/master branch!
  - Exception: None. There is no exception!
  - But what about if... No! Don't do it!
- In my experience, making changes directly to the main/master branch is the primary way Git newbies get in a pickle.
- You start on the "main" branch.
- Click on branch name to manage branches.



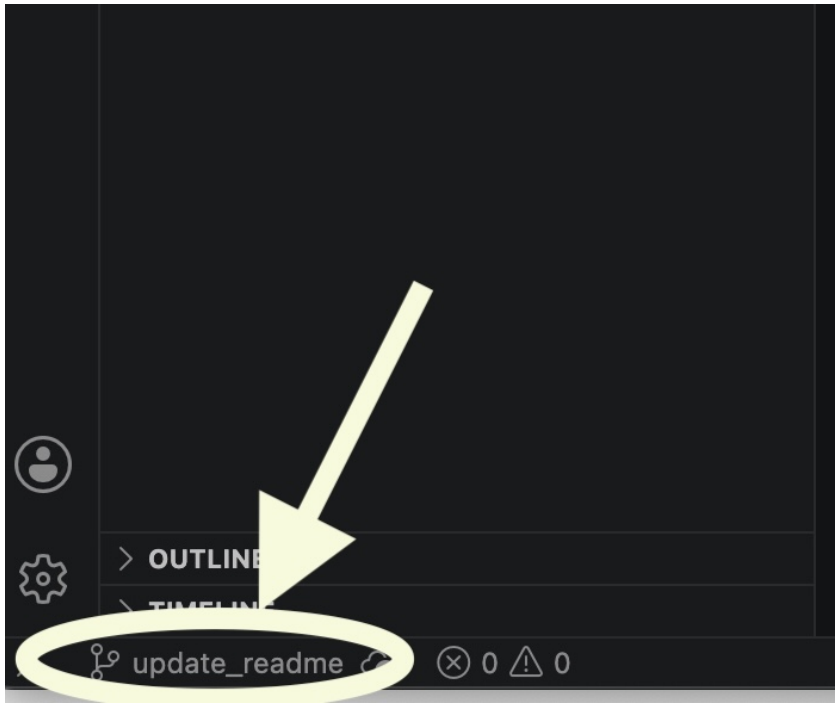
Step 1: Make a Feature Branch

- You should NEVER make changes to the main/master branch!
  - Exception: None. There is no exception!
  - But what about if... No! Don't do it!
- In my experience, making changes directly to the main/master branch is the primary way Git newbies get in a pickle.
- You start on the "main" branch.
- Click on branch name to manage branches.



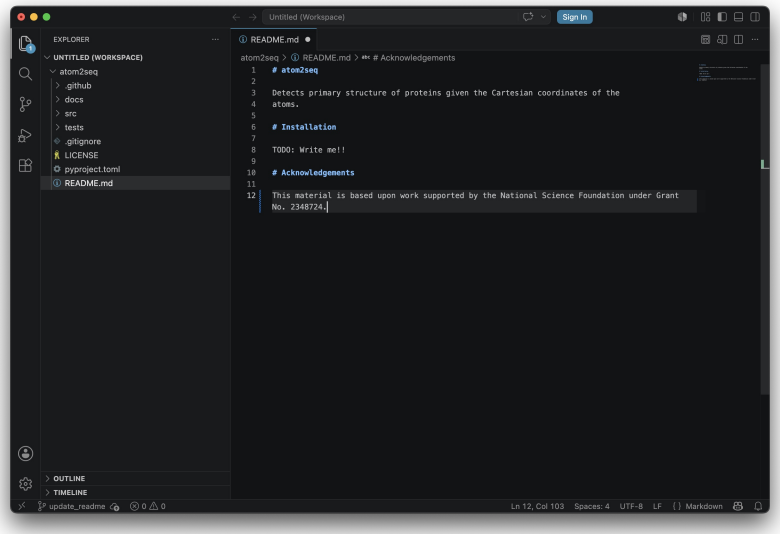
Step 1: Make a Feature Branch

- You should NEVER make changes to the main/master branch!
  - Exception: None. There is no exception!
  - But what about if... No! Don't do it!
- In my experience, making changes directly to the main/master branch is the primary way Git newbies get in a pickle.
- You start on the "main" branch.
- Click on branch name to manage branches.



Step 2: Make Some Changes

- Here we added the NSF acknowledgement.
- Make the changes you identified earlier.
- Ensure you save the file.



Check Your Understanding

How do we save the changes using git?

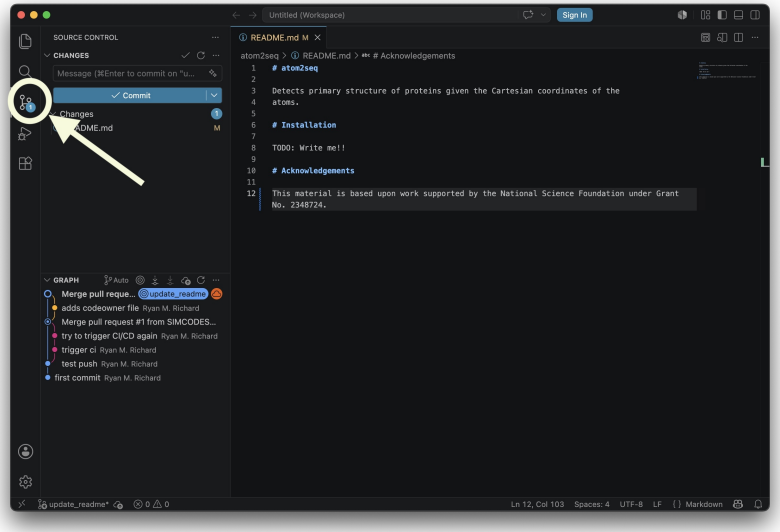
Check Your Understanding

How do we save the changes using git?

By committing them.

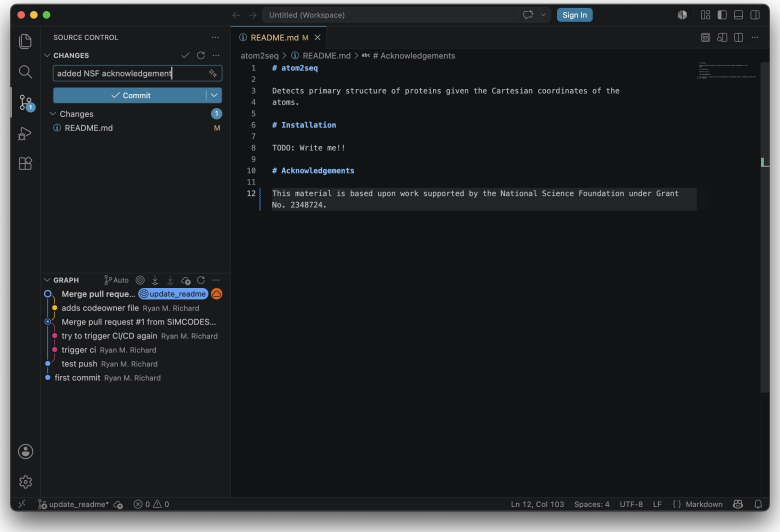
Step 3: Commit the Changes

- Switch to the VC view to see the status of all repos in your workspace.



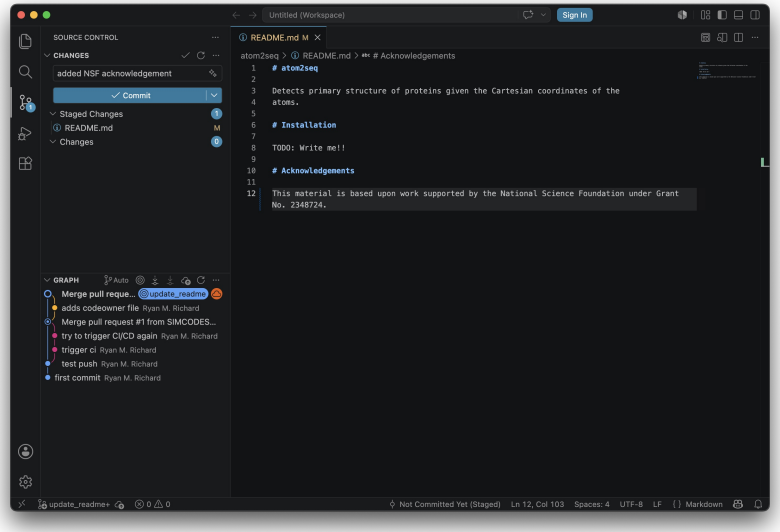
Step 3: Commit the Changes

- Switch to the VC view to see the status of all repos in your workspace.
- Write a meaningful commit message.



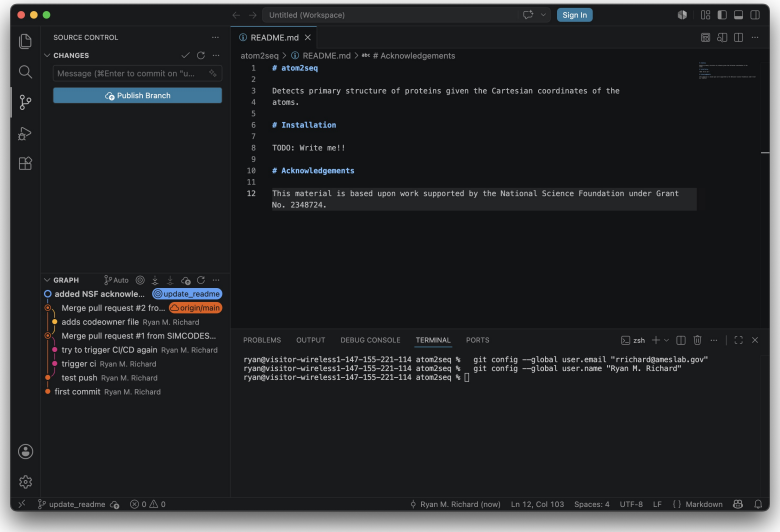
Step 3: Commit the Changes

- Switch to the VC view to see the status of all repos in your workspace.
- Write a meaningful commit message.
- Stage the changes you want to commit.



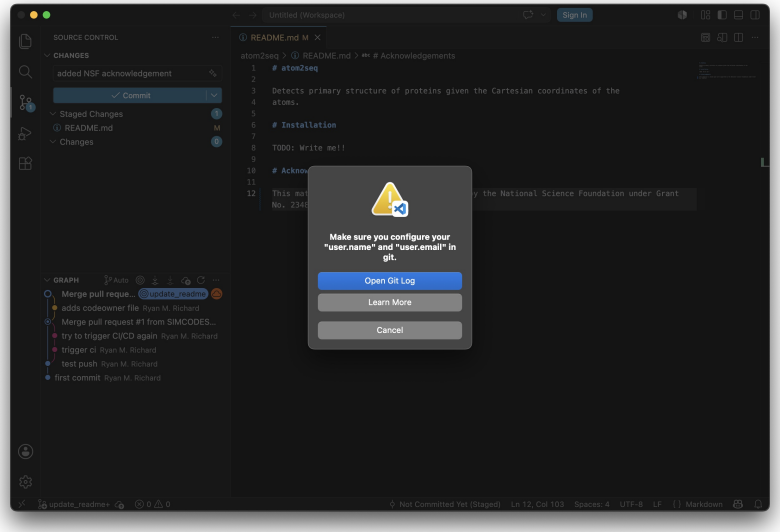
Step 3: Commit the Changes

- Switch to the VC view to see the status of all repos in your workspace.
- Write a meaningful commit message.
- Stage the changes you want to commit.
- Click “Commit”.



Aside: First commit

- If this is the very first time you have made a commit you will get an error message.



Aside: First commit

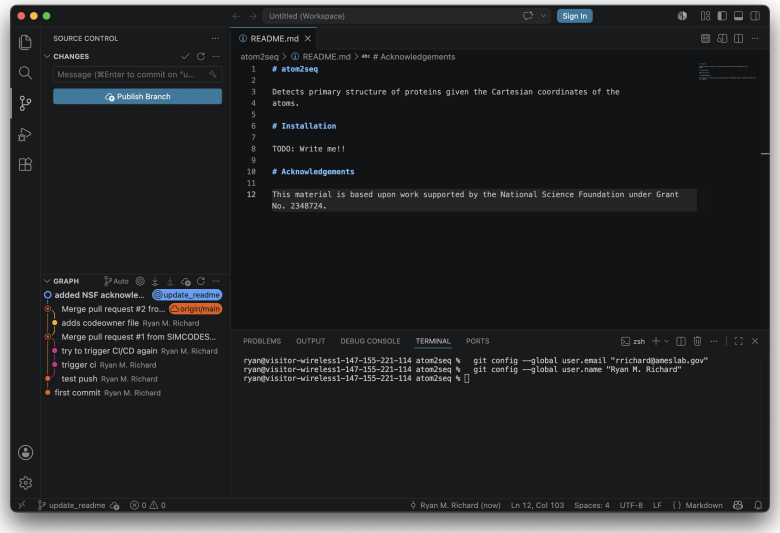
- If this is the very first time you have made a commit you will get an error message.
- Set the email and username.

Step 4: Push the Changes

- Now the changes are saved locally, but not remotely.

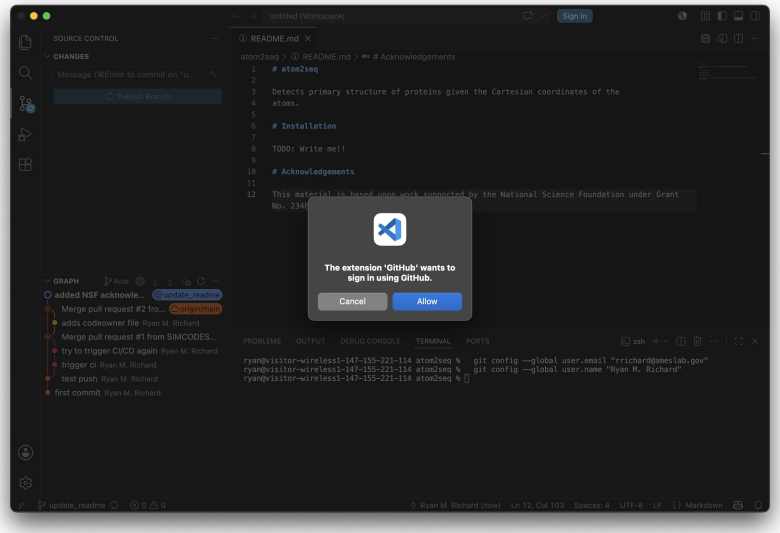
Step 4: Push the Changes

- Now the changes are saved locally, but not remotely.
- Click “Publish Branch” (first time pushing this branch) or “Synch Changes” (subsequent pushes).



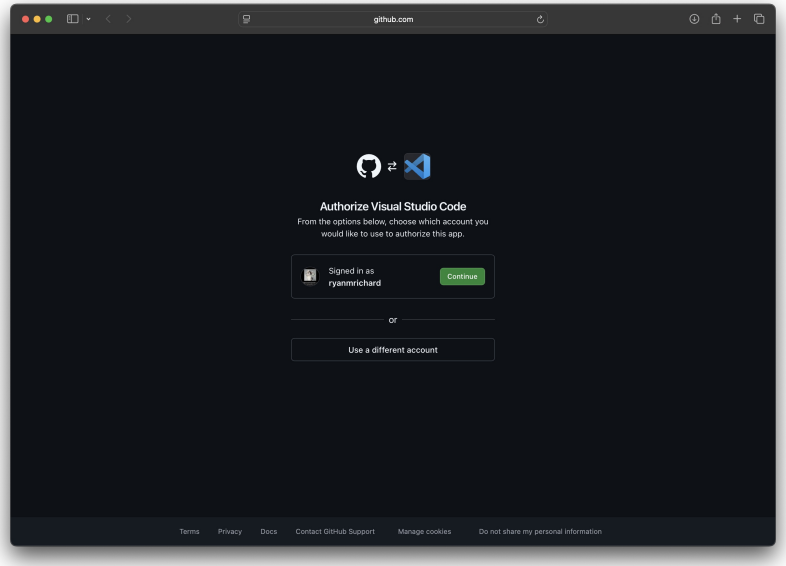
Step 4: Push the Changes

- Now the changes are saved locally, but not remotely.
- Click “Publish Branch” (first time pushing this branch) or “Synch Changes” (subsequent pushes).
- VSCode needs permissions to push to GitHub.



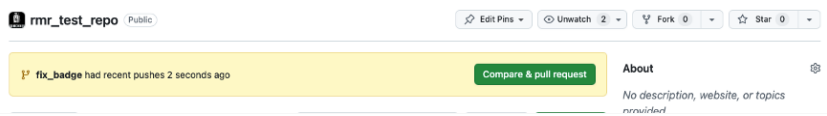
Step 4: Push the Changes

- Now the changes are saved locally, but not remotely.
- Click “Publish Branch” (first time pushing this branch) or “Synch Changes” (subsequent pushes).
- VSCode needs permissions to push to GitHub.
- Will take you to your browser so you can sign in.



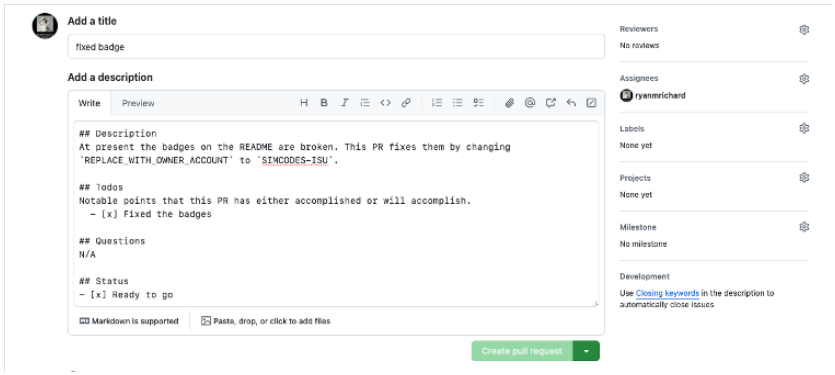
Step 5: Make a PR

- Your GitHub repo should display a “Compare & pull request” button. Click it.



Step 5: Make a PR

- Your GitHub repo should display a “Compare & pull request” button. Click it.
- You now need to fill out the next page.
  - Tell people what you did (and plan to do).
  - Pick who will review your PR (pick your mentor)
  - Assignees are working on the PR (click ”assign yourself”).
  - Ignore the rest, click “Create pull request”.
- Only done once per branch.
- If you have more changes, go back to step 2.



Step 6: Code Review

- Maintainers of the remote repo decide if your changes can/should be merged or not.
- Code review usually involves automated and human checks.
  - Automated checks run your changes and see if they work.
  - Reviewers will also look over your changes and may request changes.
- If you pass code review, the maintainers will merge your PR.
- If you do not pass code review, you go back to Step 2 to make the requested changes.

What does merging a PR accomplish?

What does merging a PR accomplish?

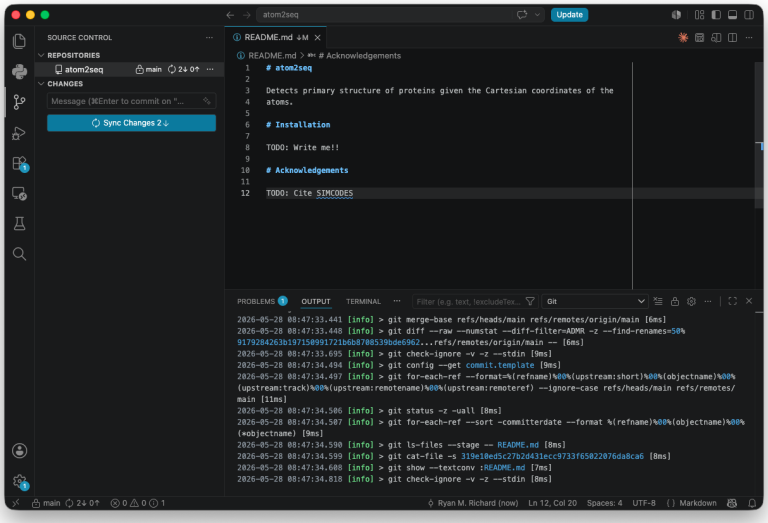
Adds your changes to the main branch

Step 7: Resetting

- Do NOT continue to keep making changes to a merged branch!
- We need to "reset" so we are again ready for step 1.

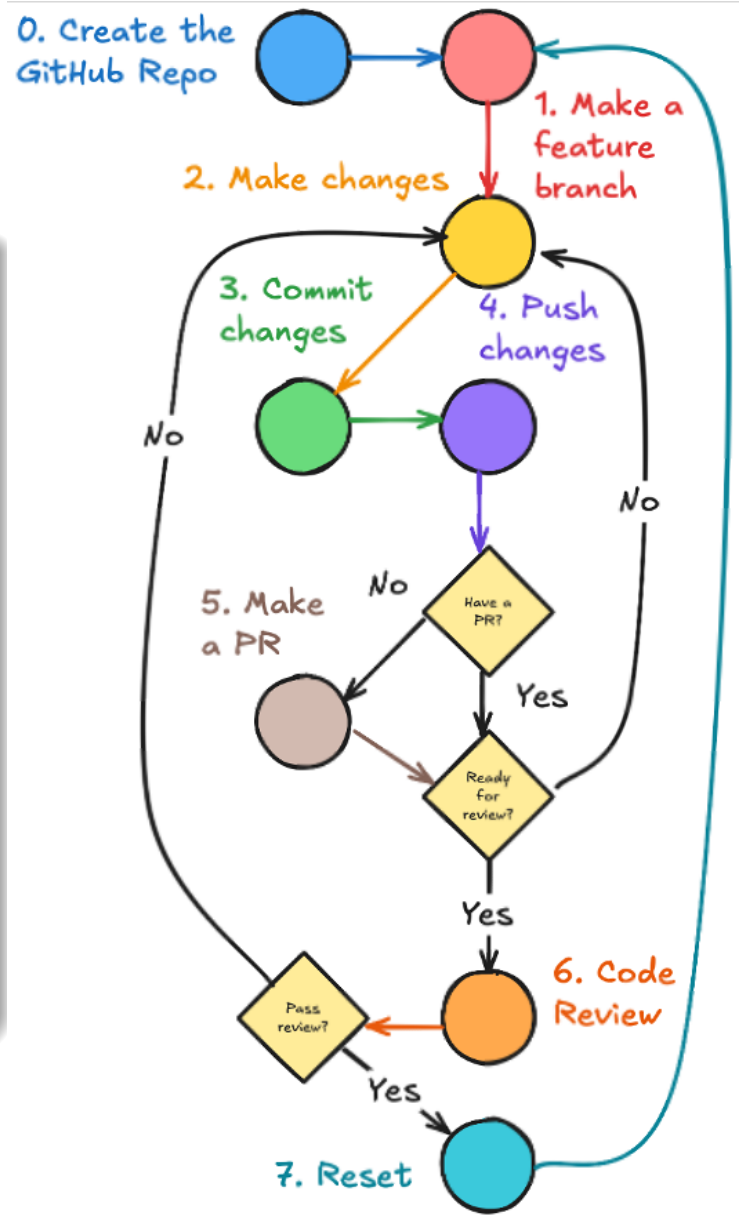
Step 7: Resetting

- Do NOT continue to keep making changes to a merged branch!
- We need to “reset” so we are again ready for step 1.
  - Switch to “main” (like Step 1, but pick “main” not create new branch).
  - In git view, click “Sync Changes”
- Skipping this step will cause histories to diverge.



Summary

- We strongly recommend following this workflow every time you make changes!
- With practice, steps 1- 4 take seconds.
- This workflow easily scales to development of multiple features.
  - Stick with a single feature when you first start out.
- NEVER skip steps 1 and 7!!!



Take Aways

- Collaborating on multiple files with multiple people is difficult.
- Version control (VC) makes the process much less painful.
  - We use Git for VC and GitHub for storing our repos.
- Software development follows the same workflow every time.
- Deviating from the workflow will increase the likelihood of conflicts, or other problems, arising from when you try to pull or push.

Acknowledgements

- MolSSI's git tutorials.
- MolSSI's GitHub tutorials.
- NSF for funding the REU.
- ISU and Ames National Lab